

ЛР 1 Job System T3

1. Цель

Внедрить в ваш движок **систему задач (job system)** и перенести на неё реальную работу движка так, чтобы эффект был **измерим** методикой с лекции 2.

Лабораторная проверяет два умения: спроектировать параллельную подсистему под свой движок и доказать её пользу цифрами.

2. Обязательный состав

2.1. Job system в движке

Варианты:

- **готовая библиотека:** enkiTS, Taskflow, marl или oneTBB (и иные), интегрированная как подсистема движка (инициализация/шатдаун, единая точка постановки задач), а не «вызов в одном месте»;
- **своя реализация** — засчитывается **только при наличии lock-free очереди или fibers**; минимальный пул на мьютексе как самостоятельное решение не засчитывается. Глубина сверх этого дополнительно поощряется в допфичах (см. 4).

2.2. Задача №1 (обязательная для всех): асинхронная загрузка ресурсов, уровень L1

Загрузка ресурсов (текстуры/модели/сцены, что есть в движке) уходит с главного потока в фон **через job system**:

- фоновое чтение файла и парсинг/декодирование (stb_image, assimp/tinyobj что у вас) задачей в job system;
- финализация (GPU-апплоад, создание буферов), **на главном потоке** через очередь-памп с лимитом на кадр
- до догрузки:
 - placeholder-ресурс
 - или загрузочный экран;
- корректный шатдаун: выход из приложения с живыми задачами загрузки не падает и не виснет.

Критерий: кадр не стоит на месте во время загрузки видно на трейсе Trasy (фриз до добавления job system -> ровный кадр после добавления).

Если async-загрузка в движке **уже есть** (std async, thread-per-resource, свой пул), обязательный вариант: **миграция на job system** + доведение до критерия L1 (памп, placeholder, шатдаун). Показать старый код как есть нельзя.

Зачем, если оно уже работает? Предмет ЛР 1 - job system, а не загрузка: если вся фоновая работа движка идёт мимо новой подсистемы, она остаётся декоративной.

У точечных механизмов есть конкретные дефекты, которые job system и исправляет:

- thread-per-resource поток на каждый ресурс (oversubscription);
- std::async не гарантирует поток на вызов, но и не даёт никакого контроля, на одной платформе это непрозрачный глобальный пул, общий с чужими задачами, на другой - свежий поток; ни там ни там нет лимита параллелизма, приоритетов (загрузка уровня важнее мипов) и единого шатдауна
- Ваш измеримый эффект здесь: "неконтролируемый параллелизм -> фиксированный пул с приоритетами, пачка ресурсов грузится быстрее и без фризов".
- Правило семестра: **один движок -> одна система фоновой работы (job system)**. Если ваш механизм уже хорош (свой пул с пампом) — задача формулируется как "обобщить его до job system", а не "переписать".

2.3. Задача №2 (на выбор)

Одна реальная задача движка, перенесённая на job system. "Реальная" = существующая работа вашего движка, не синтетический бенчмарк.

Выбор задачи — из каталога ниже, **по согласованию в чате ММ (или консультации, если требуется)**.

Кандидаты (по тому, что есть в ваших движках):

- **параллельное выполнение систем по фазам кадра.** Честная версия "параллельных систем": сначала явный анализ "кто что читает/пишет", затем read-only системы выполняются параллельно после write-фазы. Сам анализ зависимостей - часть задания и тема вопросов на приёме;
- **распараллеливание внутри одной системы:** снапшот сущностей в массив -> dispatch по диапазонам (работает и с хеш-таблицами; чтобы эффект был измерим, нужна сцена с тысячами сущностей, сделайте "замерочную");
 - сюда же как допфича: переход на плоские хеш-таблицы, concurrent и тд
- **своя физика / поиск коллизий** (broadphase, narrowphase), **только если не планируете физический middleware (Jolt/PhysX) в ЛР 4**, иначе работа выбрасывается;
- **анимация** у кого есть;
- иное по согласованию.

2.4. Замеры до/после

Методика — как на практике 2 (лекция 2):

- Release-сборка, прогрев, фиксированная сцена, N >= 3 прогонов;
- метрики: медиана + p95/p99 frame time, отдельно время целевых зон;
- трейсы Trasy "до" и "после" + таблица цифр - артефакты сдачи.

Замер в Debug, одиночный прогон или "ускорилося на глаз" - не засчитывается.

3. Критерии приёмки

Базовый вариант: **24 + 2 балла**

Блок	Баллы	Как проверяется
Наличие и полнота	12	Демо вживую: job system работает; async-загрузка L1 не фризит кадр (трейс); показана задача №2; замеры до/после по методике с цифрами и трейсами
Стабильность	6	Демо в разных сценариях: загрузка на ходу, выход с живыми задачами, повторные запуски, длинная сессия без крашей, дедлоков и зависаний. Гонки и дедлоки на демо = минус баллы этого блока
Допфичи	6	Любая работа над движком сверх обязательного (см. 4)
Своевременность	+2	Демо в свой приёмочный демо-день (25.09–02.10)

Приёмка - живое демо + презентация.

4. Допфичи (примеры, не исчерпывающе)

Глубина реализации: lock-free и fibers

Своя реализация job system засчитывается **только с lock-free очередью или fibers** (см. 2.1) — это условие зачёта, а не допфича. Дальше градация по глубине:

- work stealing (per-thread очереди); приоритеты задач / отдельный streaming-пул; зависимости между задачами - суммируются;
- минимальный пул на мьютексе без lock-free/fibers сам по себе не засчитывается ни как реализация ЛР, ни как допфича.

Job system на fibers:

- чужая реализация (FiberTaskingLib, marl) - меньше;

- своя механика ожидания через переключение - полный вес.

Async-загрузка сверх обязательного L1:

- уровень L2 - выделенный streaming-пул с ограниченным числом воркеров;
 - еще лучше реализация текстурного стриминга
- уровень L3 (только DX12\Vulkan) - upload на сору-очереди с fence: ресурс готов без участия главного потока.

Корректность и наблюдаемость:

- доказательство корректности (как у Bungie): задачи декларируют read/write-политики доступа, debug-сборка ловит пересечения до исполнения;
- разметка job-системы зонами Trasy: имена воркеров, зоны ожидания/диспетчеризации — на трейсе видно, кто чего ждёт;
- счётчики/плоты Trasy по очередям задач;
- измеримый эффект сверх одной задачи: вторая—четвёртая подсистема на задачах.

Взгляд в будущее (сквозное правило семестра): осмысленное использование вашей job system в следующих лабораторных засчитывается в колонку "допфичи" этой лабораторной. Естественные связи:

- **ЛР 2 (скриптинг):** hot reload скриптов и данных через job system;
- **ЛР 3 (рендер):** параллельная подготовка draw-данных, cook/загрузка шейдеров и мешей задачами; исполнение render graph поверх job system;
- **ЛР 4 (подсистема на выбор):** ИИ - запросы путей задачами (тема лекции 11); физика - Jolt принимает внешний JobSystem: подсунуть свою реализацию из ЛР 1 (сравнить до/после); аудио - фоновое декодирование;

Свои варианты допфичей — приветствуются, согласуйте.

5. Формат приёмки (демо-дни 25.09–02.10)

- Показ живьём: 3 минуты демо + вопросы.
- **Ротационные вопросы (АИД-2):** отвечают двое от команды — кого укажет преподаватель, не презентатор. Вопросы — по всему коду команды, не только по "своим" местам. Ответ 0/1/2, идёт в зачёт АИД-2 (итог — сумма шести лучших за семестр, пороги 6/9/11).
- Готовьте к демо: запускаемую сцену, трейсы до/после, таблицу цифр, сценарий "стресс" для блока стабильности.